

# Reviewer Pack — Arithmetic Rank of Tropical Semigroups Bounds Randomized Com...

Entry 529b07f87b43 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 04:36:46 UTC

## Arithmetic Rank of Tropical Semigroups Bounds Randomized Communication Complexity for k-CNF

Entry ID: 529b07f87b43 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 04:36:46 UTC

### 1. Conjecture

**Field A** (mathematical branch): Tropical Geometry (Tropical Semigroups) **Field B** (complexity object): Complexity Theory: Communication Complexity (k-CNF Satisfiability)

#### Statement:

{‘sentence\_1’: ‘The arithmetic rank of the tropical semigroup formed by the union of the tropicalizers of all clauses in a k-CNF instance bounds the randomized communication complexity for that instance.’, ‘sentence\_2’: ‘Specifically, if an instance  $I$  has  $n$  variables and  $m$  clauses, then the arithmetic rank  $\tau(I)$  of its corresponding tropical semigroup satisfies  $CC_R(I) \geq 2^{\tau(I)}$ .’, ‘sentence\_3’: ‘This holds for all k-CNF instances with up to 40 variables.’}

#### Rationale (proposer’s reasoning):

{‘sentence\_1’: ‘Tropical semigroups provide a geometric view of Boolean functions, and their arithmetic rank captures essential properties that are relevant to complexity.’, ‘sentence\_2’: ‘The conjecture suggests that the algebraic structure of tropical semigroups can be used to derive lower bounds on communication complexity, offering a new perspective for understanding computational problems.’, ‘sentence\_3’: ‘If true, it would demonstrate the power of tropical geometry in complexity theory and potentially lead to new methods for proving lower bounds.’}

**Taxonomy category:** COMM\_DISJ (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** d8973ea55b4719c7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if for all k-CNF instances with  $n$  variables ( $1 \leq n \leq 40$ ) and at least 30 random seeds, the arithmetic rank  $\tau(I)$  of the tropical semigroup meets  $CC_R(I) \geq 2^{\tau(I)}$  for every instance.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | UNCERTAIN | 0.80       | HITS             | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 1.00       | UNCERTAIN        | SAFE             |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "tropical geometry" AND "communication complexity" AND "k-CNF satisfiability" - "arithmetic rank" AND "tropical semigroups" AND "randomized communication complexity" - "tropicalizers" AND "k-CNF instances" AND " $CC_R(I) \geq 2^{\tau(I)}$ "

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
from fractions import Fraction

def generate_kcnf_instance(n: int, m: int) -> tuple:
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause = random.sample(variables, 2)
        clauses.append(clause)
    return len(clauses), variables, clauses

def tropicalizer(clause, variables):
    return [Fraction(abs(x)) for x in clause]

def tropical_semigroup(clauses, variables):
    semigroup = set()
    for clause in clauses:
        t_clause = tropicalizer(clause, variables)
        for i in range(len(t_clause)):
            for j in range(i + 1, len(t_clause)):
                new_element = max(t_clause[i], t_clause[j])
                if new_element not in semigroup:
                    semigroup.add(new_element)
    return semigroup

def arithmetic_rank(semigroup):
    n = len(semigroup)
    identity = Fraction(1)
    matrix = [[Fraction(0) for _ in range(n)] for _ in range(n)]
```

```

for i, x in enumerate(sorted(semigroup)):
    for j, y in enumerate(sorted(semigroup)):
        if x >= y:
            matrix[i][j] = identity

rank = 0
for row in matrix:
    if any(row):
        rank += 1
return rank

def randomized_communication_complexity(arithmetic_rank):
    return 2 ** arithmetic_rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m, variables, clauses = generate_kcnf_instance(n, n)
        semigroup = tropical_semigroup(clauses, variables)
        tau_I = arithmetic_rank(semigroup)
        CC_R_I = randomized_communication_complexity(tau_I)

        if CC_R_I < 2 ** tau_I:
            return {
                "metric_name": "Randomized Communication Complexity",
                "metric_value": CC_R_I,
                "instances_tested": m,
                "conjecture_holds": False,
                "counterexample": f"n={n}, m={m}, tau(I)={tau_I}, CC_R(I)={CC_R_I}"
            }

    results.append(CC_R_I)

    return {
        "metric_name": "Randomized Communication Complexity",
        "metric_value": sum(results) / len(results),
        "instances_tested": sum(m for _, _, m in [generate_kcnf_instance(n, n) for n in n_values]),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    support_fraction = sum(1 for r in results if r >= 2**arithmetic_rank(tropical_semigroup(genera

```

```

if all(r >= 2**arithmetic_rank(tropical_semigroup(generate_kcnf_instance(n, n)[2], generate_kcnf_instance(n, n)[2])):
    print(f"RESULT: SUPPORTED mean={mean} std=0 support_fraction=1")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean} std=0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if result < 2**arithmetic_rank(tropical_semigroup(generate_kcnf_instance(n, n)[2], generate_kcnf_instance(n, n)[2]))
    print(f"RESULT: FALSIFIED counterexample='n={n}, m={m}, tau(I)={tau_I}, CC_R(I)={CC_R_I}'")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4762ba5.py", line 95, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4762ba5.py", line 84, in run_trial
    "instances_tested": sum(m for _, _, m in [generate_kcnf_instance(n, n) for n in n_values]),
                        ~~~~~^~~~~~
TypeError: unsupported operand type(s) for +: 'int' and 'list'

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which prevents us from verifying the conjecture's support condition. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12740        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 5875         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4882         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 10397        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 22595        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10448        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15034        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11220        |

| # | Phase | Provider      | Model       | Tokens in | out | Latency (ms) |
|---|-------|---------------|-------------|-----------|-----|--------------|
| 9 | judge | ollama_remote | glm4:latest | 0         | 0   | 8071         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 101262 ms total latency. Provider mix: {'ollama\_remote': 9}

*(full prompt+response transcripts available in research/audit/529b07f87b43.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/529b07f87b43.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/529b07f87b43.tar.gz (if generated)